

A
SEMINAR REPORT
on
TERMINAL MANAGEMENT SYSTEM: A REAL-TIME IoT DEVICE
MONITORING AND CONTROL PLATFORM

Submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF TECHNOLOGY
in
ELECTRICAL ENGINEERING



Submitted By:

Ayushi Sharma
(2023UEE1337)

Submitted To:

Dr. Dipti Saxena
Professor, Department of Electrical Engineering
Malaviya National Institute of Technology
Jaipur

DEPARTMENT OF ELECTRICAL ENGINEERING
MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR
September, 2026



SHUBHSHIVALIK PRODUCTS PRIVATE LIMITED

Second Floor, 13/29-30 Room No-2&3, Rachna Building

W.E.A Ajmal Khan Road, Karol Bagh, New Delhi

Delhi – 110005

Email: info@shubhshivalik.com

CERTIFICATE

This is to certify that **Ms. Ayushi Sharma (Reg. No. 2023UEE1337)**, a student of **B.Tech in Electrical Engineering** at **Malaviya National Institute of Technology (MNIT), Jaipur**, has successfully completed her internship at **SHUBHSHIVALIK PRODUCTS** from **21.05.2026 to 08.07.2026**. During the internship, she worked on the project "**Terminal Management System (TMS) using Internet of Things (IoT)**" under the guidance of **Mr. Ashwarya Raj Singh Sisodia (G.M. – Operations)**. Her performance and conduct throughout the internship were found to be satisfactory.

We appreciate her dedication and sincere efforts and wish her every success in all her future endeavors.

This Certificate is issued for academic purpose only.


Ashutosh Singh
CHRO




MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR

CANDIDATE DECLARATION

I hereby submit the seminar report entitled “**TERMINAL MANAGEMENT SYSTEM: A REAL-TIME IoT DEVICE MONITORING AND CONTROL PLATFORM**” in the **Department of Electrical Engineering, Malaviya National Institute of Technology Jaipur**. I have worked under the supervision of **Dr. Dipti Saxena**, Professor, and declare that the work presented in this report is an authentic record of my own work and has been analysed by me, under the supervision of **Dr. Dipti Saxena**, Professor, Department of Electrical Engineering, Malaviya National Institute of Technology Jaipur.

Ayushi Sharma
2023UEE1337

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Dipti Saxena, Professor, Department of Electrical Engineering, Malaviya National Institute of Technology Jaipur, for her valuable guidance, continuous encouragement, and constructive feedback throughout the preparation of this seminar report.

I am also thankful to the Department of Electrical Engineering, MNIT Jaipur, for providing the academic environment and resources that made this work possible, and to the mentors and engineering team at Shubhshivalik Products, Noida, for the opportunity to work on the Terminal Management System project during my remote internship and for their support in understanding the practical aspects of IoT systems engineering.

Finally, I extend my heartfelt thanks to my family and friends for their constant motivation and support throughout this endeavour.

Date:

Ayushi Sharma
2023UEE1337

ABSTRACT

The growing deployment of Internet of Things (IoT) devices — sensors, actuators, gateways and controllers — across industrial and infrastructure settings has created a need for centralized platforms that can monitor device health, visualize sensor data, and issue commands in real time, while keeping a verifiable audit trail of every action taken. This report presents the design and implementation of the Terminal Management System (TMS), a full-stack IoT device control platform built during a remote internship at Shubhshivalik Products.

TMS is built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and integrates the MQTT protocol for device-to-server communication and Socket.IO (WebSockets) for server-to-browser real-time updates. The platform exposes a live dashboard showing device status, sensor readings and uptime; an interactive terminal console for sending commands to individual devices; a role-based authentication layer built on JSON Web Tokens (JWT) with three access levels; a threshold-based alerting subsystem with cooldown logic to prevent alert flooding; and an analytics module that computes device and command statistics through MongoDB aggregation pipelines. The entire stack — application services, database, MQTT broker, cache and reverse proxy — is containerized using Docker Compose for reproducible deployment.

During the internship, the report author verified the end-to-end MQTT-to-WebSocket data pipeline, corrected the Analytics dashboard to consume live aggregation endpoints in place of placeholder random data, reviewed the enforcement of role-based access control across both REST and socket layers, and validated the alert cooldown logic. The report also documents a known limitation — the Automation rules interface is presently a frontend-only mock with no backend execution engine — identified and scoped honestly during code review. This report describes the problem motivating the system, the technologies evaluated, the resulting system architecture, the implementation details of each subsystem, and the results and conclusions drawn from the work.

LIST OF CONTENTS

S. No.	TOPIC	Page No.
	Certificate	ii
	Candidate Declaration	iii
	Acknowledgements	iv
	Abstract	v
	List of Contents	vi
	List of Tables	viii
	List of Figures	ix
	List of Abbreviations	x
1.	INTRODUCTION	1
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives of the Study	1
	1.4 Scope of the Report	2
	1.5 Organisation of the Report	2
2.	TECHNOLOGY BACKGROUND	3
	2.1 Internet of Things and Device Management Platforms	3
	2.2 MQTT Protocol for IoT Messaging	3
	2.3 WebSockets and Real-Time Communication	3
	2.4 The MERN Stack	4
	2.5 Containerization with Docker	4
	2.6 Summary	5
3.	SYSTEM DESIGN AND ARCHITECTURE	6
	3.1 System Overview	6
	3.2 System Architecture	6
	3.3 Database Design	7
	3.4 Real-Time Communication Design	8
	3.5 Authentication and Role-Based Access Control Design	8
	3.6 Alerting Subsystem Design	9
4.	IMPLEMENTATION	10
	4.1 Technology Stack	10

S. No.	TOPIC	Page No.
	4.2 Backend Implementation	10
	4.3 REST API Implementation	11
	4.4 Frontend Implementation	12
	4.5 Analytics Implementation	12
	4.6 Deployment Using Docker Compose	13
	4.7 Contribution During the Internship	13
5.	RESULTS AND CONCLUSION	15
	5.1 Testing and Results	15
	5.2 Limitations	15
	5.3 Conclusion	16
	5.4 Future Scope	16
	APPENDIX — Selected Code Listings	18
	REFERENCES	19

LIST OF TABLES

S. No.	Title of Table	Page No.
3.1	Key Fields of the Device Schema	7
3.2	Key Fields of the CommandLog Schema	7
4.1	Technology Stack Used in TMS	10
4.2	REST API Endpoints Exposed by the Backend	11
4.3	MQTT Topics Used by TMS	12
4.4	Docker Compose Services and Ports	13
5.1	Manual Test Cases and Observed Results	15

LIST OF FIGURES

S. No.	Name of Figure	Page No.
3.1	System Architecture of TMS	6
3.2	Real-Time Sensor Data Flow (MQTT → Backend → Dashboard)	8
4.1	Docker Compose Multi-Container Deployment	13

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
MQTT	Message Queuing Telemetry Transport
ODM	Object Data Modelling
QoS	Quality of Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
TMS	Terminal Management System
UI	User Interface
WS	WebSocket

CHAPTER 1

INTRODUCTION

1.1 Background

The Internet of Things (IoT) refers to networks of physical devices — sensors, actuators, gateways, controllers and cameras — that collect data from their environment and, in many cases, act upon it. As organizations deploy larger fleets of such devices across factories, warehouses and utility infrastructure, the challenge shifts from simply connecting a device to managing it: knowing whether it is online, what it is reporting, whether its readings are within a safe operating range, and who issued the last command to it. This class of problem is addressed by device management or terminal management platforms, which sit between the physical devices and the human operators who monitor and control them.

This report documents the Terminal Management System (TMS), a full-stack IoT device control platform built as part of a remote internship at Shubhshivalik Products, Noida, undertaken between May and July 2026. TMS was built using the MERN stack (MongoDB, Express.js, React.js and Node.js) together with the MQTT protocol for device communication and Socket.IO for real-time updates to the browser. The report author's role during the internship was to extend, debug and verify this platform against its stated feature set, working directly with the existing codebase.

1.2 Problem Statement

Organizations operating distributed IoT devices commonly face four recurring problems:

- Devices are geographically or logically scattered, and there is no single live view of which devices are online, offline, or in an error state.
- Reading sensor values or issuing commands to a device typically requires connecting to it individually (for example, over SSH or a vendor-specific tool), which does not scale.
- There is no standard audit trail recording who issued which command, to which device, and whether it succeeded — this becomes important when diagnosing faults after the fact.
- There is no automated mechanism to flag a sensor reading that has crossed a safe operating threshold, so unsafe conditions can go unnoticed until a device fails.

TMS is designed to address each of these four problems within a single, unified web dashboard.

1.3 Objectives of the Study

The objectives set out for this seminar report and the underlying project work are as follows:

- To study the design of a real-time IoT device management platform built on the MERN stack, MQTT and WebSockets, and to document its system architecture.
- To examine how the platform implements authentication, role-based access control, threshold-based alerting, and command audit logging.
- To describe the implementation of the platform's backend services, REST API, frontend dashboard, and containerized deployment.
- To report on the specific contributions made during the internship — verifying the real-time data pipeline, correcting the analytics dashboard to use live data, and reviewing security and alerting logic — along with the limitations identified in the existing codebase.

1.4 Scope of the Report

The scope of this report is limited to the TMS platform as it existed at the end of the internship period (July 2026). It covers the backend services (Express REST API, MQTT integration, WebSocket server, MongoDB data layer), the React-based frontend dashboard, the Docker Compose-based deployment topology, and the specific engineering contributions made by the report author. The report does not cover integration with physical hardware devices beyond the simulated device client included in the codebase, nor does it cover the Automation rules feature, which is documented in Chapter 4 as a known frontend-only limitation rather than a completed feature.

1.5 Organisation of the Report

The remainder of this report is organized as follows. Chapter 2 reviews the technology background relevant to the platform — IoT device management, the MQTT protocol, WebSocket-based real-time communication, the MERN stack, and containerization with Docker. Chapter 3 presents the system design and architecture of TMS, including its database schema, real-time communication design, and security design. Chapter 4 describes the implementation of each subsystem and details the specific contributions made during the internship. Chapter 5 presents the testing carried out, the limitations identified, and the conclusions and future scope of the work.

CHAPTER 2

TECHNOLOGY BACKGROUND

2.1 Internet of Things and Device Management Platforms

IoT device management platforms are responsible for device registration, status monitoring, command dispatch, and telemetry collection at scale. Common design patterns in such platforms include a message broker that decouples devices from the management server, a time-series or document store for sensor history, a rules or alerting engine that reacts to incoming data, and a dashboard for human operators. TMS follows this general pattern: devices communicate through an MQTT broker, sensor history and device state are stored in MongoDB, an alert service reacts to threshold breaches, and a React dashboard provides the operator-facing view.

2.2 MQTT Protocol for IoT Messaging

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency networks, which makes it a common choice for IoT communication in place of heavier request-response protocols such as HTTP. In the publish-subscribe model, devices publish messages to hierarchical topics (for example, `tms/devices/DEV-A1/sensor`) without needing to know which clients, if any, are listening; a broker such as Eclipse Mosquitto is responsible for routing each published message to every client subscribed to a matching topic pattern.

MQTT defines three Quality of Service (QoS) levels that trade off delivery guarantees against overhead: QoS 0 (at most once, no acknowledgement), QoS 1 (at least once, acknowledged, possible duplicates) and QoS 2 (exactly once, higher overhead). TMS subscribes to its device topics at QoS 1, which guarantees that sensor readings and status updates are delivered at least once even if a connection is briefly interrupted, at the cost of occasionally processing a duplicate message.

2.3 WebSockets and Real-Time Communication

While MQTT is well suited to device-to-server communication, web browsers do not natively speak MQTT in a way that scales cleanly to many concurrent dashboard users. WebSockets provide a persistent, full-duplex connection between a browser and a server over a single TCP connection, allowing the server to push updates to the client the instant new data is available, rather than the client having to poll repeatedly. Socket.IO is a widely used library that

implements WebSocket communication (with automatic fallback and reconnection handling) and adds convenient abstractions such as named events and rooms. TMS uses Socket.IO on both the Express backend and the React frontend, so that events such as `device:sensor`, `device:status` and `device:alert` are pushed to every connected dashboard the moment they occur on the backend.

2.4 The MERN Stack

The MERN stack refers to four JavaScript-based technologies used together to build full-stack web applications:

- **MongoDB** — a document-oriented NoSQL database, used here through the Mongoose object data modelling (ODM) library, which provides schema definition, validation and query building over MongoDB's flexible document model.
- **Express.js** — a minimal web application framework for Node.js, used to implement the REST API, middleware pipeline (authentication, rate limiting, error handling) and HTTP server.
- **React.js** — a component-based JavaScript library for building user interfaces, used with functional components and hooks to build the dashboard, terminal console and analytics views.
- **Node.js** — a JavaScript runtime built on Chrome's V8 engine, used to run the Express server, the MQTT client, and the Socket.IO server within a single process.

Using a single language (JavaScript) across the client, server and database access layer reduces context-switching during development and allows data structures (such as a sensor reading object) to be shared conceptually between the frontend and backend without a serialization boundary.

2.5 Containerization with Docker

Docker packages an application together with its runtime dependencies into a portable container image, which behaves identically regardless of the host machine it runs on. Docker Compose extends this to multi-service applications, allowing several containers — for example, an application server, a database, and a message broker — to be defined declaratively in a single YAML file and started together with one command, each container isolated from the others but able to communicate over a shared network. This is well suited to a platform such as TMS, which depends on several independent services (MongoDB, an MQTT broker, a cache, a reverse

proxy) that would otherwise each need to be installed and configured by hand on every machine the application runs on.

2.6 Summary

This chapter reviewed the technologies that TMS is built on: MQTT as the device communication protocol, WebSockets (via Socket.IO) as the mechanism for pushing real-time updates to the browser, the MERN stack as the application framework, and Docker Compose as the deployment mechanism. Chapter 3 uses these building blocks to present the system design and architecture of TMS.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

TMS is organized around four functional pillars: Monitor (live device status and sensor data on a dashboard), Control (an interactive terminal console for sending commands), Protect (JWT authentication with role-based access control) and Alert (per-device configurable threshold alerts). Each pillar is realized through a combination of the backend services described below and the corresponding frontend pages: Dashboard, Devices, Terminal, Analytics, Automation and Settings.

3.2 System Architecture

Figure 3.1 shows the high-level architecture of TMS. IoT devices publish status and sensor readings to an Eclipse Mosquitto MQTT broker over topics of the form `tms/devices/{deviceId}/{status|sensor|command|response}`. The Express backend subscribes to all device topics through `mqtt.service.js`, parses each incoming message, persists it to MongoDB, and re-broadcasts it to every connected browser using Socket.IO. The React dashboard never communicates with the MQTT broker directly; it receives all live updates over its WebSocket connection to the backend, which acts as the single integration point between the device-facing and browser-facing halves of the system.

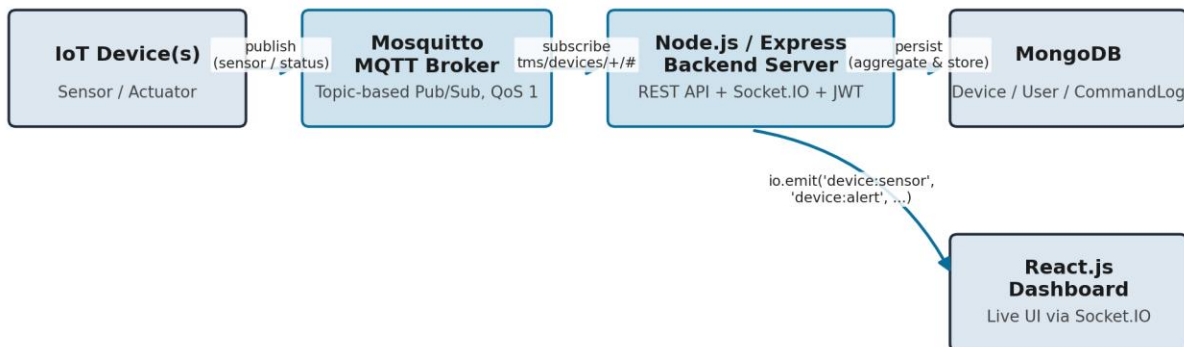


Figure 3.1 System Architecture of TMS

3.3 Database Design

TMS uses MongoDB with three primary Mongoose schemas: User, Device and CommandLog. The User schema stores name, email, a bcrypt-hashed password, a role (admin, operator or

viewer) and an isActive flag. Table 3.1 and Table 3.2 summarize the key fields of the Device and CommandLog schemas, which carry most of the platform's operational data.

Field	Type	Purpose
deviceId	String (unique)	Human-readable unique identifier for the device, used in MQTT topics
name / type	String / Enum	Display name and device category: sensor, actuator, gateway, controller, camera, relay
status	Enum	online offline error idle, updated on every MQTT status message
protocol	Enum	MQTT HTTP CoAP WebSocket — communication protocol used by the device
sensorReadings[]	Embedded array	Rolling buffer of the last 100 readings (type, value, unit, timestamp)
alertThresholds	Embedded object	Per-metric min / max thresholds (e.g. temperature, humidity) used by the alert service
ownedBy	ObjectId (ref: User)	Reference to the user who registered the device

Table 3.1 Key Fields of the Device Schema

Field	Type	Purpose
deviceId / device	String / ObjectId (ref: Device)	Target device, stored both as a readable ID and a document reference
sentBy	ObjectId (ref: User)	The user who issued the command, populated with name and email on read
command / payload	String / Mixed	The command name and an arbitrary JSON payload
status	Enum	sent acknowledged failed timeout
executionTime	Number (ms)	Time taken to publish the command, used by the analytics aggregation

Table 3.2 Key Fields of the CommandLog Schema

The Device schema keeps only the most recent 100 sensor readings per device using MongoDB's \$push operator combined with a \$slice: -100 modifier, which bounds the size of each device document instead of allowing an unbounded array to grow indefinitely. Longer-term analytics are instead computed on demand through aggregation pipelines (see Section 4.5) rather than by retaining full history inside the document.

3.4 Real-Time Communication Design

Figure 3.2 traces a single sensor reading through the system, from the moment a device publishes it to the moment it appears on a dashboard. This design keeps MQTT (device-facing) and Socket.IO (browser-facing) as two independent channels that only meet at the backend, which is deliberately the single place where a reading is validated, persisted and checked against alert thresholds before being broadcast onward.

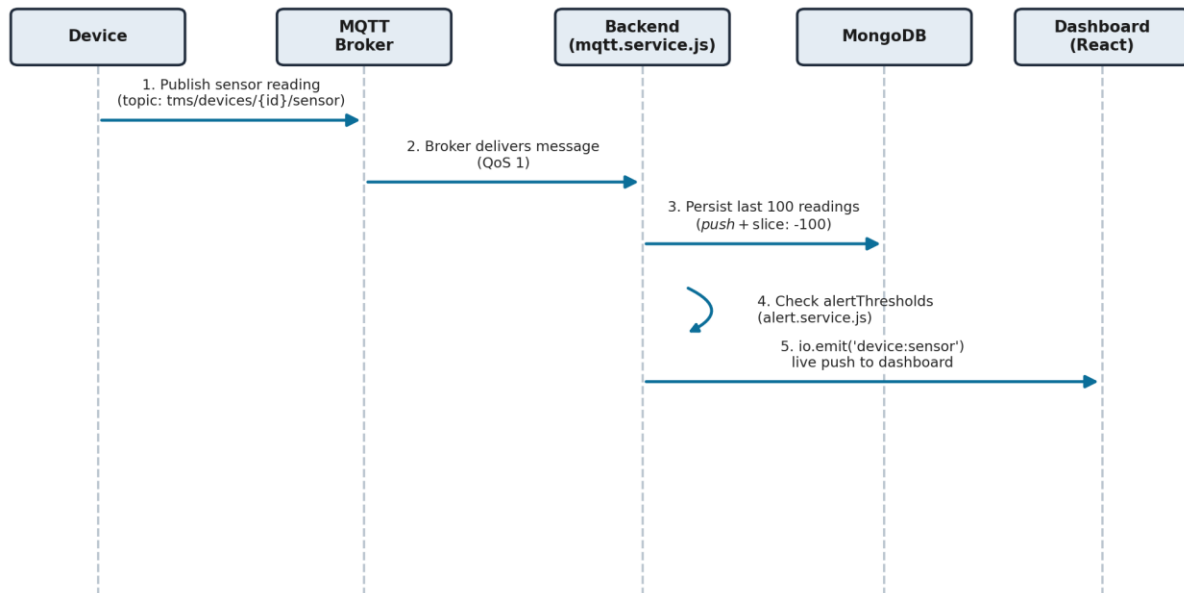


Figure 3.2 Real-Time Sensor Data Flow (MQTT → Backend → Dashboard)

This separation has a practical benefit for reliability: if every dashboard client disconnects, the MQTT subscription and MongoDB writes continue unaffected, so no sensor history is lost while no one is watching. Conversely, a dashboard client can reconnect and immediately request the latest state through the REST API without needing to replay the MQTT stream.

3.5 Authentication and Role-Based Access Control Design

Every user is assigned one of three roles at account creation: admin (full access, including device and threshold management), operator (can send commands and acknowledge alerts) or viewer (read-only access to the dashboard, analytics and history). Access control is enforced identically on two independent surfaces:

- REST routes — a `protect()` middleware verifies the JWT bearer token on the Authorization header, loads the corresponding user from MongoDB, and attaches it to the request; a further `authorize(...roles)` middleware restricts specific routes (for example, device deletion) to one or more roles.

- WebSocket connections — the same JWT is verified during the Socket.IO handshake using an `io.use()` middleware, so an unauthenticated client cannot open a socket connection at all, not merely be denied specific events.

Passwords are never stored or compared in plain text: the User schema hashes incoming passwords with `bcrypt` at a cost factor of 12 inside a Mongoose pre-save hook, and authentication compares the candidate password against the stored hash using `bcrypt`'s constant-time comparison.

3.6 Alerting Subsystem Design

Each device stores its own `alertThresholds` object (minimum and maximum values per metric, such as temperature or humidity). Whenever a batch of sensor readings arrives, `alert.service.js` checks every reading against the matching threshold for that device and metric. A breach constructs a human-readable message (for example, “DEV-A1: temperature HIGH (31.2 > 28)”) and is only broadcast if no alert for the same device and metric has fired within the previous 60 seconds — this cooldown, tracked in an in-memory map keyed by `deviceId:metric`, prevents a reading that oscillates around a threshold from flooding the dashboard with repeated alerts. Valid alerts are broadcast to all connected clients through `io.emit('device:alert', ...)`, the same channel used for ordinary sensor updates.

CHAPTER 4

IMPLEMENTATION

4.1 Technology Stack

Table 4.1 summarizes the technology stack used to implement TMS, organized by layer.

Layer	Technologies
Frontend	React.js 18 (Hooks), React Router v6, Socket.IO-client, Recharts, Axios, Context API
Backend	Node.js, Express.js, Socket.IO, MQTT.js, MongoDB + Mongoose, JWT, bcryptjs, express-rate-limit, Helmet
Infrastructure	Docker & Docker Compose, Eclipse Mosquitto, Nginx (reverse proxy), MongoDB 6, Redis 7

Table 4.1 Technology Stack Used in TMS

4.2 Backend Implementation

The backend entry point, `server.js`, wires together an Express application, an HTTP server, and a Socket.IO server on the same port. On startup it connects to MongoDB, initializes the WebSocket layer (`initWebSocket`) and the MQTT layer (`initMQTT`), and mounts four route groups — `/api/auth`, `/api/devices`, `/api/commands` and `/api/analytics` — behind a global rate limiter (100 requests per 15 minutes per client) and Helmet security headers.

MQTT integration (`mqtt.service.js`)

On connection, the service subscribes to all four device topic patterns at QoS 1. Its message handler parses the topic to extract the device ID and event type, then branches on that type: a status message updates the device's status, lastSeen and uptime fields in MongoDB and re-emits `device:status` over Socket.IO; a sensor message appends the new readings to the device's rolling buffer (capped at 100 entries) and emits `device:sensor`; a response message is relayed directly as `command:response`. A separate `publishCommand(deviceId, command, payload)` function is exposed for the command controller to use when dispatching an operator's command back out to a device.

WebSocket layer (`websocket.service.js`)

The Socket.IO server authenticates every incoming connection using a JWT passed in the socket handshake before accepting it, exactly as the REST middleware does for HTTP requests. Once

connected, a client can join a device-specific room (subscribe:device) to scope updates, and can trigger a demo data stream (demo:start / demo:stop) that emits synthetic sensor readings every two seconds — used during development and demonstrations when no physical or simulated device is publishing over MQTT.

4.3 REST API Implementation

Table 4.2 lists the REST endpoints exposed by the backend. All routes other than registration and login require a valid JWT, and routes that mutate state are further restricted by role using the `authorize()` middleware described in Section 3.5.

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new user account	Public
POST	/api/auth/login	Authenticate and receive a JWT	Public
GET	/api/auth/me	Fetch the current authenticated user	Any role
GET	/api/devices	List devices, with optional status / type / group filters	Any role
POST	/api/devices	Register a new device	admin, operator
GET	/api/devices/:id	Fetch a single device by deviceId	Any role
PUT	/api/devices/:id/toggle	Toggle a device on / off	admin, operator
DELETE	/api/devices/:id	Remove a device	admin
POST	/api/commands/send	Publish a command to a device over MQTT	admin, operator
GET	/api/commands/history	Fetch the command audit log	Any role
GET	/api/analytics/sensors	Historical sensor readings for a device	Any role
GET	/api/analytics/devices	Aggregated device counts by status / type	Any role
GET	/api/analytics/commands	Aggregated command statistics	Any role

Table 4.2 REST API Endpoints Exposed by the Backend

Table 4.3 lists the MQTT topic patterns the backend subscribes to and publishes on; the `+` wildcard matches any single topic level, so `tms/devices/+/sensor` matches the sensor topic of every registered device.

Topic Pattern	Direction	Purpose
tms/devices/+/status	Device → Backend	Device online/offline status and uptime
tms/devices/+/sensor	Device → Backend	Batched sensor readings (type, value, unit)

Topic Pattern	Direction	Purpose
tms/devices/+/command	Backend → Device	Commands published to a specific device
tms/devices/+/response	Device → Backend	Command acknowledgement / response payload

Table 4.3 MQTT Topics Used by TMS

4.4 Frontend Implementation

The React frontend is organized into reusable components (DeviceCard, TerminalConsole, SensorChart, AlertPanel, CommandBuilder, Navbar) and route-level pages (Dashboard, Devices, Terminal, Analytics, Automation, Settings), with global state managed through AuthContext and DeviceContext. A custom useWebSocket hook wraps the Socket.IO client connection, so that any component can subscribe to live device events without duplicating connection-management logic. Axios is used for REST calls through a shared api.js utility that attaches the JWT to every request.

4.5 Analytics Implementation

The Analytics page is backed by three MongoDB aggregation pipelines exposed through the /api/analytics routes. Device statistics group all devices by their type field using a single \$group stage to produce live counts of online, offline and error devices. Command statistics compute the ten busiest devices by command volume using \$group followed by \$sort and \$limit, and separately compute the average command execution time using \$avg over the executionTime field recorded on every CommandLog entry, for example:

```
CommandLog.aggregate([
  { $match: { executionTime: { $exists: true } } },
  { $group: { _id: null, avg: { $avg: "$executionTime" } } }
]);
```

Computing these statistics inside MongoDB, rather than fetching raw documents and reducing them in application code, keeps the computation close to the data and avoids transferring entire collections over the network for what is ultimately a handful of summary numbers.

4.6 Deployment Using Docker Compose

Figure 4.1 shows the six services defined in docker-compose.yml and the ports each exposes. Running docker-compose up --build starts every service in its own container on a shared bridge network, with the backend depending on MongoDB, Mosquitto and Redis being ready, and

Nginx depending on both the frontend and backend — matching the intended production topology on a single development machine.

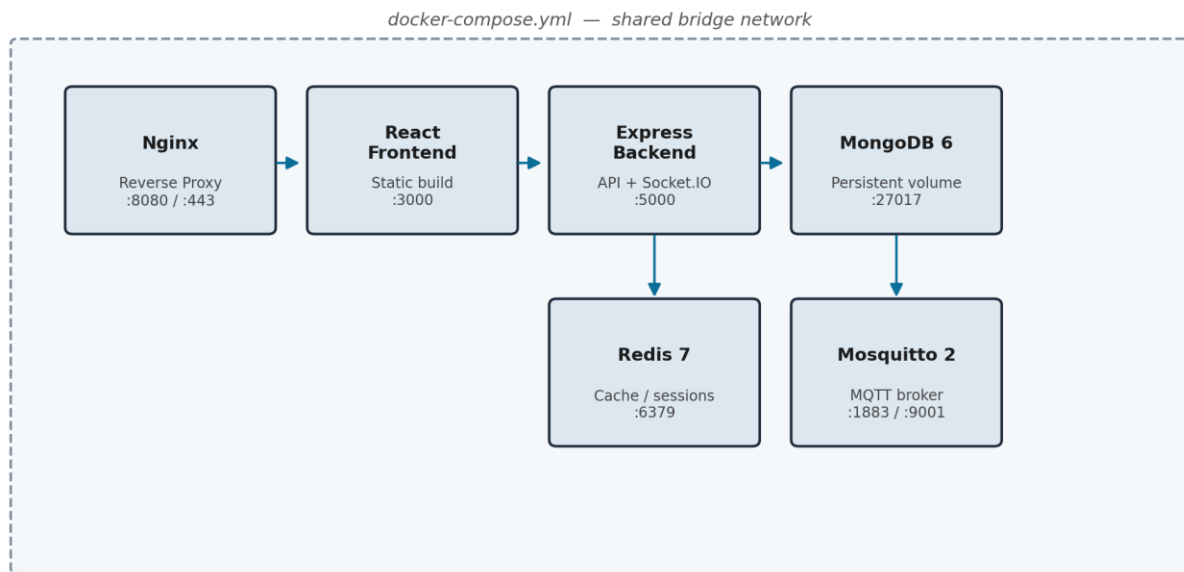


Figure 4.1 Docker Compose Multi-Container Deployment

Service	Image / Build	Port(s)	Role
mongodb	mongo:6	27017	Primary data store, persistent volume
mosquitto	eclipse-mosquitto:2	1883 / 9001	MQTT broker for device communication
redis	redis:7-alpine	6379	Cache / session store
backend	Custom (Node.js)	5000	Express REST API + Socket.IO server
frontend	Custom (Nginx-served React build)	3000	Static React production build
nginx	nginx:alpine	8080 / 443	Reverse proxy in front of frontend and backend

Table 4.4 Docker Compose Services and Ports

4.7 Contribution During the Internship

The codebase for TMS existed prior to the internship; the work carried out during the internship consisted of extending, verifying and correcting it. The specific contributions made were as follows.

Wiring the Analytics dashboard to real data

On inspection, the Analytics page was found to be rendering values generated by `Math.random()` on every render, despite the backend already exposing working `/api/analytics` endpoints backed by the aggregation pipelines described in Section 4.5. This was corrected by replacing the placeholder logic with calls to the real endpoints, so that the charts on the Analytics page reflect actual device and command data stored in MongoDB rather than synthetic values.

Verifying the end-to-end MQTT ↔ WebSocket pipeline

Using the `simulated-device.js` script included in the backend, the full pipeline described in Section 3.4 was traced and tested manually: a simulated device publishing sensor readings over MQTT, through topic parsing and MongoDB persistence in `mqtt.service.js`, through the threshold check in `alert.service.js`, to the resulting `device:sensor` and `device:alert` events rendering live on the dashboard.

Reviewing RBAC and alert-cooldown logic

The role checks described in Section 3.5 were reviewed for consistency across both the REST routes (`authorize()` middleware) and the Socket.IO handshake (`io.use()` middleware), confirming that a viewer-role account could not invoke command-sending routes even if it obtained a valid token. The 60-second alert cooldown described in Section 3.6 was validated against a reading oscillating around a threshold, confirming that repeated breaches within the cooldown window are suppressed rather than re-broadcast.

Scoping the Automation feature honestly

The Automation page presents a rules-based “IF trigger THEN action” interface backed entirely by local React component state (`useState`). On review, no corresponding backend execution engine was found — rules created in the UI are not evaluated against live sensor data or dispatched as commands. This was documented as a known limitation of the current codebase rather than represented as a working feature, consistent with the principle of only claiming functionality that is actually wired end-to-end.

CHAPTER 5

RESULTS AND CONCLUSION

5.1 Testing and Results

The platform was tested manually against the feature set described in Chapters 3 and 4, using the simulated device client to generate MQTT traffic and multiple browser sessions with different user roles to exercise the RBAC and real-time update paths. Table 5.1 summarizes the principal test cases and their observed outcomes.

Test Case	Expected / Observed Outcome	Result
User login with valid / invalid credentials	JWT issued only for correct credentials; 401 on failure	Pass
Viewer role attempts POST /api/commands/send	Request rejected with 403 Insufficient permissions	Pass
Simulated device publishes sensor reading over MQTT	Reading visible on dashboard within ~1 second, persisted in MongoDB	Pass
Sensor reading crosses configured max threshold	device:alert broadcast once; repeat breach within 60 s suppressed	Pass
Analytics page load after real-endpoint integration	Displays live aggregation results instead of random values	Pass
Unauthenticated Socket.IO connection attempt	Handshake rejected before a socket connection is established	Pass
docker-compose up --build (cold start)	All six containers reach a healthy / running state	Pass

Table 5.1 Manual Test Cases and Observed Results

In every case tested, the observed behaviour matched the design described in Chapter 3. The correction to the Analytics page (Section 4.7) was verified by comparing the values shown on the dashboard against a manual query of the same aggregation pipeline run directly against the MongoDB instance, confirming the two matched.

5.2 Limitations

- The Automation rules interface (Section 4.7) is presently a frontend-only mock: rules can be created and toggled in the UI, but no backend engine evaluates them against live data or dispatches the corresponding actions.

- Testing was carried out using the simulated device client rather than physical hardware; while the MQTT topic structure is intended to be protocol-compatible with real devices, no physical device integration was verified during the internship.
- The alert cooldown window (60 seconds) and the sensor buffer size (100 readings per device) are presently hard-coded rather than being configurable per deployment.
- Redis is provisioned in the Docker Compose stack for caching and session storage but is not yet consumed by any backend route in the current codebase.

5.3 Conclusion

This report has described the design and implementation of the Terminal Management System, a full-stack IoT device control platform combining MQTT-based device communication with a WebSocket-driven real-time dashboard, JWT-based role access control, threshold alerting, and MongoDB aggregation-based analytics, deployed as a six-service Docker Compose stack. Working within an existing codebase during the internship, the report author verified the real-time MQTT-to-dashboard pipeline end to end, corrected the Analytics page to consume live aggregation data in place of placeholder values, reviewed the enforcement of role-based access control across both the REST and WebSocket layers, validated the alert cooldown logic, and identified and documented the Automation feature's current limitation rather than overstating its functionality.

The exercise reinforced two practical lessons beyond the specific technologies used. First, MQTT and WebSockets serve complementary rather than competing roles in an IoT backend — one is suited to constrained devices, the other to browsers — and a clean architecture keeps the two channels independent, meeting only at a single, well-defined point on the server. Second, verifying a codebase critically, by tracing what is actually wired end-to-end rather than trusting what the user interface appears to do, was essential to producing an honest account of the platform's capabilities.

5.4 Future Scope

- Implement a backend rules-evaluation engine so that Automation rules created in the UI are actually evaluated against incoming sensor data and dispatched as commands.
- Integrate physical IoT hardware (e.g. an ESP32 or Raspberry Pi publishing over MQTT) to validate the platform beyond the simulated device client.

- Make the alert cooldown window and sensor buffer size configurable per device or per deployment, rather than hard-coded constants.
- Use the already-provisioned Redis instance for session caching and for rate-limiting state, reducing repeated MongoDB lookups on the authentication path.
- Add automated integration tests covering the MQTT → backend → WebSocket pipeline, in place of the manual testing carried out in this report.

APPENDIX — SELECTED CODE LISTINGS

This appendix reproduces two short excerpts from the TMS backend that illustrate the design points discussed in Chapters 3 and 4.

A.1 MQTT Sensor Message Handling (backend/services/mqtt.service.js)

```
if (eventType === "sensor") {
  await Device.findOneAndUpdate(
    { deviceId },
    {
      $push: {
        sensorReadings: {
          $each: payload.readings,
          $slice: -100, // keep last 100 readings
        },
      },
      lastSeen: new Date(),
    }
  );
  io.emit("device:sensor", { deviceId, readings: payload.readings });
}
```

A.2 Threshold Check with Alert Cooldown (backend/services/alert.service.js)

```
if (alertMsg) {
  const key = `${deviceId}:${reading.type}`;
  const lastAlert = activeAlerts.get(key);
  const now = Date.now();
  if (!lastAlert || now - lastAlert > 60000) {
    activeAlerts.set(key, now);
    io.emit("device:alert", {
      deviceId, type: "threshold", severity: "warning",
      message: alertMsg, timestamp: new Date(), reading,
    });
  }
}
```

REFERENCES

- [1] OASIS, "MQTT Version 5.0," OASIS Standard, 2019. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [2] Eclipse Foundation, "Eclipse Mosquitto — An open source MQTT broker," Eclipse Mosquitto Documentation. Available: <https://mosquitto.org/documentation/>
- [3] Socket.IO, "Socket.IO Documentation," Socket.IO. Available: <https://socket.io/docs/v4/>
- [4] Express.js, "Express — Node.js web application framework," Express Documentation. Available: <https://expressjs.com/>
- [5] MongoDB Inc., "Aggregation Pipeline," MongoDB Manual. Available: <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>
- [6] Mongoose, "Mongoose ODM v8 Documentation," Mongoose.js. Available: <https://mongoosejs.com/docs/>
- [7] React, "React Documentation," React.dev. Available: <https://react.dev/>
- [8] Docker Inc., "Docker Compose Overview," Docker Documentation. Available: <https://docs.docker.com/compose/>
- [9] IETF, "RFC 7519 — JSON Web Token (JWT)," Internet Engineering Task Force, 2015. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [10] OWASP Foundation, "REST Security Cheat Sheet," OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html